

ROBO_NEU – Ablauf für verteilte BO-Benchmarks auf der LRZ Compute Cloud

Stand: 05.08.2026

1 Überblick

ROBO_NEU rechnet BO-Benchmarks (Bayessche Optimierung auf BBOB/COCO-Funktionen) über mehrere unabhängige LRZ-Compute-Cloud-VMs parallel. Jede VM verarbeitet ihren eigenen Task-Chunk mit mehreren Worker-Prozessen. Es gibt **keine zentrale Steuerung** – jede VM wird einzeln angesprochen.

Begriff	Bedeutung	Hinweis
VM = OpenStack Instance	1 VM entspricht immer genau 1 Instance	Anzahl über LRZ-Kontingent (Quota) begrenzt
instance_0 ... instance_N	Nur Ordernamen für Task-Chunks in ROBO_NEU	Kein Bezug zum OpenStack-Begriff
Worker	Parallele Prozesse pro VM	Richtwert: 1 Worker pro vCPU

2 Ablaufdiagramm

```
Dein PC (prepare_tasks.py, split_tasks.py)
|
+-- scp instance_0 --> VM 0 -- SSH -- run_instance.py --workers N
+-- scp instance_1 --> VM 1 -- SSH -- run_instance.py --workers N
+-- scp instance_2 --> VM 2 -- SSH -- run_instance.py --workers N
+-- scp instance_3 --> VM 3 -- SSH -- run_instance.py --workers N
|
Dein PC <-- scp status.json / results.csv zurueck von jeder VM <--+
|
merge_results.py --> results_gesamt.xlsx
```

3 Schritt-für-Schritt-Anleitung

3.1 Schritt 1 – Einmalig zentral (lokal oder auf VM 0)

Erzeugt die vollständige Task-Liste und teilt sie in Chunks auf, einen pro VM.

```
python prepare_tasks.py
python split_tasks.py --instanzen 4
```

Ergebnis: data/instances/instance_0 ... instance_3, jeweils mit tasks.csv (eigener Anteil) und tensor_noise.npy (identische Kopie).

3.2 Schritt 2 – Pro VM: Code + Daten hochladen

Für jede VM einmal wiederholen (Code z. B. per `git clone`, Daten per `scp/rsync`):

```
scp -r data/instances/instance_0 user@vm0-ip:~/robo_neu/  
scp -r data/instances/instance_1 user@vm1-ip:~/robo_neu/  
scp -r data/instances/instance_2 user@vm2-ip:~/robo_neu/  
scp -r data/instances/instance_3 user@vm3-ip:~/robo_neu/
```

3.3 Schritt 3 – Pro VM: einloggen und Prozess starten

Jede VM bekommt eine eigene SSH-Session. `tmux` sorgt dafür, dass der Prozess bei SSH-Abbruch weiterläuft:

```
ssh user@vm0-ip  
cd ~/robo_neu  
tmux new -s robo  
python run_instance.py --instance-dir instance_0 --workers 10  
# Strg+B, D zum Detachen der tmux-Session
```

Dasselbe wiederholen für `vm1/instance_1`, `vm2/instance_2`, `vm3/instance_3` – jede VM arbeitet unabhängig, ohne Wissen von den anderen.

3.4 Schritt 4 – Fortschritt prüfen (jederzeit während des Laufs möglich)

```
scp user@vm0-ip:~/robo_neu/instance_0/results/status.json data/instances/instance_0/  
results/  
# ... fuer alle VMs wiederholen  
python check_progress.py --pattern "data/instances/instance_*/results/status.json"
```

3.5 Schritt 5 – Nach Abschluss: Ergebnisse zusammenführen

```
scp user@vm0-ip:~/robo_neu/instance_0/results/results.csv data/instances/instance_0/  
results/  
# ... fuer alle VMs wiederholen  
python merge_results.py --pattern "data/instances/instance_*/results/results.csv" --  
out data/results_gesamt.xlsx
```

4 Wichtige Hinweise

- Bei Absturz oder SSH-Abbruch: denselben `run_instance.py`-Befehl einfach erneut ausführen – bereits erledigte Tasks (`status=ok` in `results.csv`) werden automatisch übersprungen.
- Fehlerhafte einzelne Tasks stoppen den Gesamtlauf nicht (`status=error` wird protokolliert, Rest läuft weiter).
- Worker-Anzahl sollte der Anzahl vCPUs der jeweiligen VM entsprechen – mehr Worker als Kerne bringt keinen Geschwindigkeitsvorteil.
- Mehr VMs als aktuell 4 sind möglich, erfordern aber ggf. eine Quota-Erhöhung beim LRZ (vCPU-/Instanz-Kontingent des Compute-Cloud-Projekts).
- `config.py` ist aktuell auf Testwerte reduziert – für den echten Lauf (500.000+ Kombinationen) `FUNKTIONEN`, `DIMENSIONEN`, `SURROGATE_AKQUISITION_PAARE` und `N_SAMPLE_INIT` wieder hochskalieren.

5 Offene Automatisierungsmöglichkeit (optional)

Da Schritt 2–4 pro VM identisch sind, lässt sich der Ablauf mit einem kleinen Wrapper-Skript (Bash/PowerShell) über eine Liste von VM-IPs automatisieren, statt jede VM manuell einzeln anzusteuern – auf Wunsch kann ein solches Skript ergänzt werden.